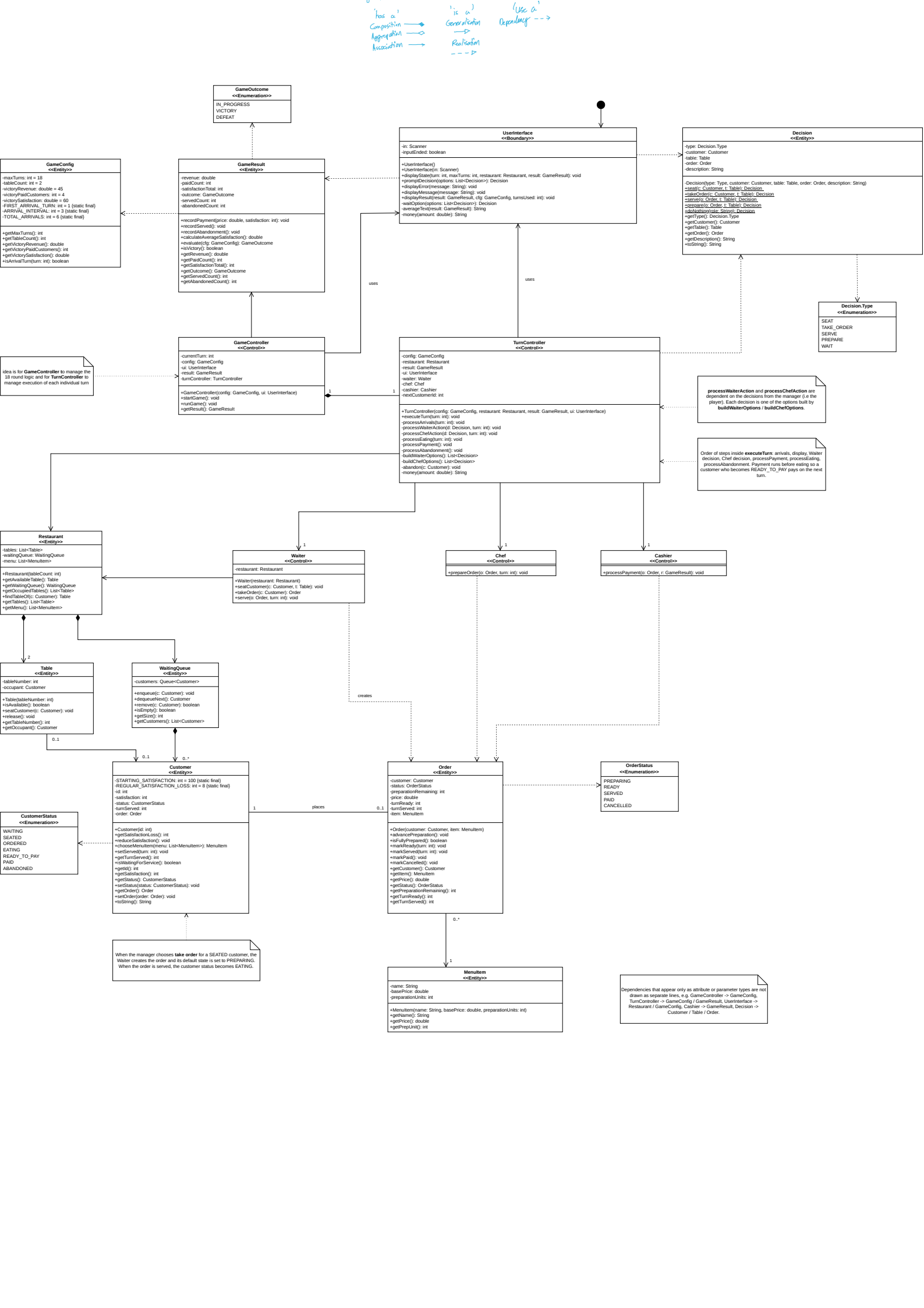

'has a'
Composition
Aggregation
Association
'is a'
Generalization
Realization
(Use a)
Dependency
GameOutcome
<<Enumeration>>
IN_PROGRESS
VICTORY
DEFEAT
UserInterface
<<Boundary>>
-in: Scanner
-inputEnded: boolean
+UserInterface()
+UserInterface(in: Scanner)
+displayState(turn: int, maxTurns: int, restaurant: Restaurant, result: GameResult): void
+promptDecision(options: List<Decision>): Decision
+displayError(message: String): void
+displayMessage(message: String): void
+displayResult(result: GameResult, cfg: GameConfig, turnsUsed: int): void
-waitOption(options: List<Decision>): Decision
-averageText(result: GameResult): String
-money(amount: double): String
Decision
<<Entity>>
-type: Decision.Type
-customer: Customer
-table: Table
-order: Order
-description: String
-Decision(type: Type, customer: Customer, table: Table, order: Order, description: String)
+seat(c: Customer, t: Table): Decision
+takeOrder(c: Customer, t: Table): Decision
+serve(o: Order, t: Table): Decision
+prepare(o: Order, t: Table): Decision
+doNothing(role: String): Decision
+getType(): Decision.Type
+getCustomer(): Customer
+getTable(): Table
+getOrder(): Order
+getDescription(): String
+toString(): String
Decision.Type
<<Enumeration>>
SEAT
TAKE_ORDER
SERVE
PREPARE
WAIT
GameConfig
<<Entity>>
-maxTurns: int = 18
-tableCount: int = 2
-victoryRevenue: double = 45
-victoryPaidCustomers: int = 4
-victorySatisfaction: double = 60
-FIRST_ARRIVAL_TURN: int = 1 {static final}
-ARRIVAL_INTERVAL: int = 3 {static final}
-TOTAL_ARRIVALS: int = 6 {static final}
+getMaxTurns(): int
+getTableCount(): int
+getVictoryRevenue(): double
+getVictoryPaidCustomers(): int
+getVictorySatisfaction(): double
+isArrivalTurn(turn: int): boolean
GameResult
<<Entity>>
-revenue: double
-paidCount: int
-satisfactionTotal: int
-outcome: GameOutcome
-servedCount: int
-abandonedCount: int
+recordPayment(price: double, satisfaction: int): void
+recordServed(): void
+recordAbandonment(): void
+calculateAverageSatisfaction(): double
+evaluate(cfg: GameConfig): GameOutcome
+isVictory(): boolean
+getRevenue(): double
+getPaidCount(): int
+getSatisfactionTotal(): int
+getOutcome(): GameOutcome
+getServedCount(): int
+getAbandonedCount(): int
uses
uses
GameController
<<Control>>
-currentTurn: int
-config: GameConfig
-ui: UserInterface
-result: GameResult
-turnController: TurnController
+GameController(config: GameConfig, ui: UserInterface)
+startGame(): void
+runGame(): void
+getResult(): GameResult
TurnController
<<Control>>
-config: GameConfig
-restaurant: Restaurant
-result: GameResult
-ui: UserInterface
-waiter: Waiter
-chef: Chef
-cashier: Cashier
-nextCustomerId: int
+TurnController(config: GameConfig, restaurant: Restaurant, result: GameResult, ui: UserInterface)
+executeTurn(turn: int): void
-processArrivals(turn: int): void
-processWaiterAction(d: Decision, turn: int): void
-processChefAction(d: Decision, turn: int): void
-processEating(turn: int): void
-processPayment(): void
-processAbandonment(): void
-buildWaiterOptions(): List<Decision>
-buildChefOptions(): List<Decision>
-abandon(c: Customer): void
-money(amount: double): String
1
1
idea is for GameController to manage the 18 round logic and for TurnController to manage execution of each individual turn
processWaiterAction and processChefAction are dependent on the decisions from the manager (i.e the player). Each decision is one of the options built by buildWaiterOptions / buildChefOptions.
Order of steps inside executeTurn: arrivals, display, Waiter decision, Chef decision, processPayment, processEating, processAbandonment. Payment runs before eating so a customer who becomes READY_TO_PAY pays on the next turn.
Restaurant
<<Entity>>
-tables: List<Table>
-waitingQueue: WaitingQueue
-menu: List<MenuItem>
+Restaurant(tableCount: int)
+getAvailableTable(): Table
+getWaitingQueue(): WaitingQueue
+getOccupiedTables(): List<Table>
+findTableOf(c: Customer): Table
+getTables(): List<Table>
+getMenu(): List<MenuItem>
Waiter
<<Control>>
-restaurant: Restaurant
+Waiter(restaurant: Restaurant)
+seatCustomer(c: Customer, t: Table): void
+takeOrder(c: Customer): Order
+serve(o: Order, turn: int): void
Chef
<<Control>>
+prepareOrder(o: Order, turn: int): void
Cashier
<<Control>>
+processPayment(o: Order, r: GameResult): void
1
1
1
2
Table
<<Entity>>
-tableNumber: int
-occupant: Customer
+Table(tableNumber: int)
+isAvailable(): boolean
+seatCustomer(c: Customer): void
+release(): void
+getTableNumber(): int
+getOccupant(): Customer
WaitingQueue
<<Entity>>
-customers: Queue<Customer>
+enqueue(c: Customer): void
+dequeueNext(): Customer
+remove(c: Customer): boolean
+isEmpty(): boolean
+getSize(): int
+getCustomers(): List<Customer>
creates
0..1
0..1
0..*
Customer
<<Entity>>
-STARTING_SATISFACTION: int = 100 {static final}
-REGULAR_SATISFACTION_LOSS: int = 8 {static final}
-id: int
-satisfaction: int
-status: CustomerStatus
-turnServed: int
-order: Order
+Customer(id: int)
+getSatisfactionLoss(): int
+reduceSatisfaction(): void
+chooseMenuItem(menu: List<MenuItem>): MenuItem
+setServed(turn: int): void
+getTurnServed(): int
+isWaitingForService(): boolean
+getId(): int
+getSatisfaction(): int
+getStatus(): CustomerStatus
+setStatus(status: CustomerStatus): void
+getOrder(): Order
+setOrder(order: Order): void
+toString(): String
1
places
0..1
Order
<<Entity>>
-customer: Customer
-status: OrderStatus
-preparationRemaining: int
-price: double
-turnReady: int
-turnServed: int
-item: MenuItem
+Order(customer: Customer, item: MenuItem)
+advancePreparation(): void
+isFullyPrepared(): boolean
+markReady(turn: int): void
+markServed(turn: int): void
+markPaid(): void
+markCancelled(): void
+getCustomer(): Customer
+getItem(): MenuItem
+getPrice(): double
+getStatus(): OrderStatus
+getPreparationRemaining(): int
+getTurnReady(): int
+getTurnServed(): int
OrderStatus
<<Enumeration>>
PREPARING
READY
SERVED
PAID
CANCELLED
CustomerStatus
<<Enumeration>>
WAITING
SEATED
ORDERED
EATING
READY_TO_PAY
PAID
ABANDONED
When the manager chooses take order for a SEATED customer, the Waiter creates the order and its default state is set to PREPARING. When the order is served, the customer status becomes EATING.
0..*
1
MenuItem
<<Entity>>
-name: String
-basePrice: double
-preparationUnits: int
+MenuItem(name: String, basePrice: double, preparationUnits: int)
+getName(): String
+getPrice(): double
+getPrepUnit(): int
Dependencies that appear only as attribute or parameter types are not drawn as separate lines, e.g. GameController -> GameConfig, TurnController -> GameConfig / GameResult, UserInterface -> Restaurant / GameConfig, Cashier -> GameResult, Decision -> Customer / Table / Order.